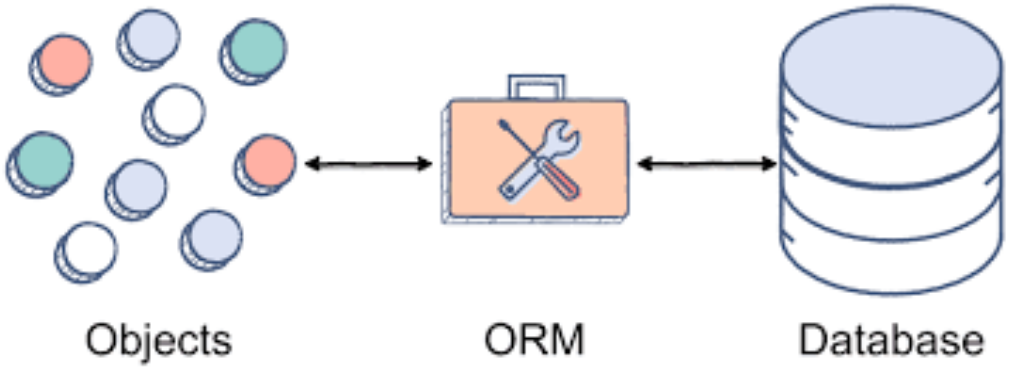
Object-Relational Mapper (ORM)

Part 1



14.04.26

Intro to SQL & SQLite

Table of Content

Why Not Just Use Files?

So far, we have stored data in text files and CSV files. This works fine for small amounts of data, but has problems at scale:

- Searching becomes slow and inefficient
- Inserting and deleting is cumbersome
- No built-in protection against data corruption
- Hard to handle relationships between data

Businesses often manage millions of records. They need something better.

What Is a DBMS?

Database Management System (DBMS): Software specifically designed to store, retrieve, and manipulate large amounts of data in an organized and efficient manner.

Your application never touches the data directly. Instead:

Benefits:

- The programmer does not need to worry about how data is physically stored
- No need to implement search or sort algorithms yourself
- The DBMS handles all reading, writing, and searching

Introduction to SQL (Structured Query Language)

The **standard language** for working with databases.

With SQL, we can:

CRUD Operation	Row-Level Example	Table-Level Example
Create	**INSERT** INTO users (...) VALUES (...)	**CREATE** TABLE users (...)
Read	**SELECT** * FROM users	**SHOW** CREATE TABLE users or querying system catalogs
Update	**UPDATE** users SET ... WHERE ...	**ALTER** TABLE users ADD COLUMN ...
Delete	**DELETE** FROM users WHERE ...	**DROP** TABLE users

Think of SQL like asking questions:

```
SELECT name, email FROM customers WHERE age > 25
```

"Show me the names and emails of customers which are older than 25"

SQLite: A lightweight, file-based relational database system

Unlike larger systems (MySQL, PostgreSQL, Oracle), SQLite:

- Stores the entire database in a single file on your computer
- Requires no server or configuration
- Is perfect for learning, testing, and small applications
- Comes built-in with Python (no installation needed)


```
import sqlite3
```

When to use SQLite	When NOT to use SQLite
• Learning databases • Desktop applications • Mobile apps (Android, iOS) • Small web applications	• Large-scale web applications with many concurrent users • Systems requiring advanced security features • Applications needing remote database access

Tables, Rows, and Columns

A database is organized into **tables** like an excel spreadsheet. See SQLite Viewer App

Column (Field): A category of data

- id, name, email, age
- Each column has a specific data type (INTEGER, TEXT, etc.)

Row (Record): One complete entry

- Alice Smith's record is one row
- Bob Jones's record is another row

Primary Key: A unique identifier for each row

- In this table, id is the primary key
- No two rows can have the same id
- Allows us to locate specific records

id	name	email	age
1	Alice Smith	alice@example.com	28
2	Bob Jones	bob@example.com	35
3	Carol White	carol@example.com	42

customers | products | employees | +

Data Types in SQLite

Every column in a table has a data type. SQLite supports several common types (Datatypes In SQLite):

Data Type	Description	Examples
INTEGER	Whole numbers	42, -5, 0, 1000
REAL	Decimal numbers	3.14, 99.99, -2.5
TEXT	Text strings	"Alice", "hello@example.com"
BLOB	Binary data	images, files
NULL	Missing or unknown value	

Why data types matter:

- The database can enforce rules (e.g., age must be a number)
- Queries can filter and sort correctly
- Storage is more efficient

Example: The customers table has these types:

id → INTEGER (whole number)
name → TEXT (string)
email → TEXT (string)
age → INTEGER (whole number)

id	name	email	age
1	Alice Smith	alice@example.com	28
2	Bob Jones	bob@example.com	35
3	Carol White	carol@example.com	42

customers | products | employees | +

Primary Keys

A primary key is a column which identifies a unique record in a table.

The value must be unique for each row

Examples:

- Student ID number
- Serial number of a product
- Invoice number

Datum **18.07.2018**
Kunde **10018**
Rechnung **2018328**

Sehr geehrter Herr Mustermann,

nachfolgend berechnen wir Ihnen wie vorab besprochen:

Rechnung 2018328.
Das Rechnungsdatum entspricht dem Leistungsdatum

Art-Nr.	Bezeichnung	Menge	Einzelpreis	Betrag
10001	Außenfassade streichen, inkl. Materialkosten	8,5 Std	131,59	1.118,52 €

Nettobetrag	939,93 €
Umsatzsteuer 19%	178,59 €
Rechnungsbetrag	**1.118,52 €**

Vielen Dank für Ihren Auftrag!

Identity Column

Sometimes the data does not naturally contain a unique key. In this case an identify column can be used. Identity columns contain automatically generated auto-incrementing integers.

Using the **id** uniquely identifies each record in the table.

id	name	email	age
1	Alice Smith	alice@example.com	28
2	Bob Jones	bob@example.com	35
3	Carol White	carol@example.com	42
4	Carol White	carol@email.com	46

customers | products | employees | +

NULL-Values

When a column contains no value, it will be NULL. NULL means there is no data for the specific record in the column.

id	name	email	age
1	Alice Smith	alice@example.com	NULL
2	Bob Jones	bob@example.com	NULL
3	Carol White	NULL	42
4	NULL	carol@email.com	46

customers | products | employees | +

Connecting to an SQLite Database in Python

To work with SQLite in Python, we use the sqlite3 module (built-in, no installation needed).

```python
import sqlite3

# Connect to a database file
# If the file doesn't exist, SQLite creates it
conn = sqlite3.connect("company.db")

# Create a cursor (like a "pen" to write queries)
cursor = conn.cursor()

# Execute a query
cursor.execute("SELECT * FROM employees")

# Fetch results
results = cursor.fetchall()

# Always close the connection when done
conn.close()
```

Key points:

- `sqlite3.connect()` opens or creates a database file
- `cursor` is how we send commands to the database
- `execute()` runs an SQL command
- `fetchall()` retrieves the result
- `close()` the connection when finished

File-based advantage:

- The database lives in `company.db` on your computer
- You can copy, backup, or email this single file

Creating Tables

Before we can store data, we need to create the table with columns and data types.

SQL Syntax

```
CREATE TABLE employees (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    email TEXT,
    salary REAL,
    hireDate TEXT
)
```

Python Example

```
import sqlite3

conn = sqlite3.connect("company.db")
cursor = conn.cursor()

cursor.execute("""
    CREATE TABLE employees (
        id INTEGER PRIMARY KEY,
        name TEXT NOT NULL,
        email TEXT,
        salary REAL,
        hireDate TEXT
    )
""")

conn.commit()  # Save changes to the database
conn.close()
```

Reading Tables

If we want to see how the table was created, we can use **SHOW CREATE TABLE**. Outputs the original CREATE TABLE statement that was used to define the table

SQL Syntax

```
SHOW CREATE TABLE employees;
```

Output

```
CREATE TABLE employees (
   id INTEGER PRIMARY KEY,
   name TEXT NOT NULL,
   email TEXT,
   salary REAL,
   hireDate TEXT
)
```

Updating Tables

We can update the table with the **ALTER TABLE** command

- Add new column ADD COLUMN
- Rename column RENAME COLUMN
- Rename table RENAME TO

Important limitations in SQLite (plan your table structure carefully):

- **Cannot delete a column** (some other databases allow this)
- **Cannot change a column's data type** (some other databases allow this)
- **Cannot add NOT NULL columns to existing tables** without a default value

SQL Syntax

```
ALTER TABLE employees RENAME COLUMN hireDate TO hire_date;
```

Python Example

```
import sqlite3


conn = sqlite3.connect("database.db")
cursor = conn.cursor()

cursor.execute("ALTER TABLE employees RENAME COLUMN hireDate TO hire_date")

conn.commit()
conn.close()
```

Deleting Tables

Use the **DROP** command to delete tables

SQL Syntax

```
DROP TABLE employees;
```

Python Example

```
import sqlite3

conn = sqlite3.connect("company.db")
cursor = conn.cursor()

# Delete the entire employees table
cursor.execute("DROP TABLE employees")

conn.commit()  # Save changes
conn.close()
```

Deleting Tables

Important to know when deleting

- `DROP TABLE` removes the entire table and **ALL** its data
- This action cannot be undone
- Use with extreme caution in production environments

Check if table exists first:

```
cursor.execute("""DROP TABLE IF EXISTS employees""")
```

The **`IF EXISTS`** clause prevents an error if the table doesn't exist. This is a safe practice when writing scripts that might run multiple times.

When to use DROP TABLE:	When NOT to use:
• Cleaning up during development and testing • Resetting data for a fresh start • Removing tables that are no longer needed	• On production databases without backups • Without understanding the consequences

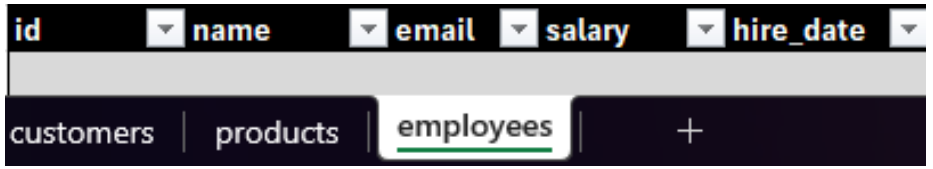
Inserting Data with INSERT

SQL Syntax

```
INSERT INTO employees (id, name, email, salary, hire_date)
VALUES (1, 'Alice Smith', 'alice@company.com', 75000, '2023-01-15')
```

Python Example

```
import sqlite3
conn = sqlite3.connect("company.db")
cursor = conn.cursor()
# Insert one record
cursor.execute("""
   INSERT INTO employees (id, name, email, salary, hire_date)
   VALUES (1, 'Alice Smith', 'alice@company.com', 75000, '2023-01-15')
""")
# Insert another record
cursor.execute("""
   INSERT INTO employees (id, name, email, salary, hire_date)
   VALUES (2, 'Bob Jones', 'bob@company.com', 65000, '2023-03-20')
""")
conn.commit()  # Save all changes
conn.close()
```



id	name	email	salary	hire_date
1	Alice Smith	alice@company.com	75000	2023-01-15
2	Bob Jones	bob@company.com	65000	2023-03-20

customers | products | employees | +

Querying Data with SELECT

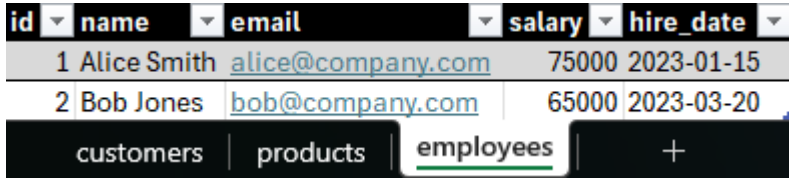
The **SELECT** statement retrieves data from a table. One of the most important SQL commands.

Basic SQL Syntax

```
SELECT column(s) FROM table_name
```

SQL Syntax – Get everything (*) in employees

```
SELECT * FROM employees (* = all columns)
```



id	name	email	salary	hire_date
1	Alice Smith	alice@company.com	75000	2023-01-15
2	Bob Jones	bob@company.com	65000	2023-03-20

customers | products | employees | +

Python Example

```python
import sqlite3

conn = sqlite3.connect('company.db')
cursor = conn.cursor()

# Retrieve all columns from all rows
cursor.execute("SELECT * FROM employees")
results = cursor.fetchall()

for row in results:
    print(row)

conn.close()
```

Output:

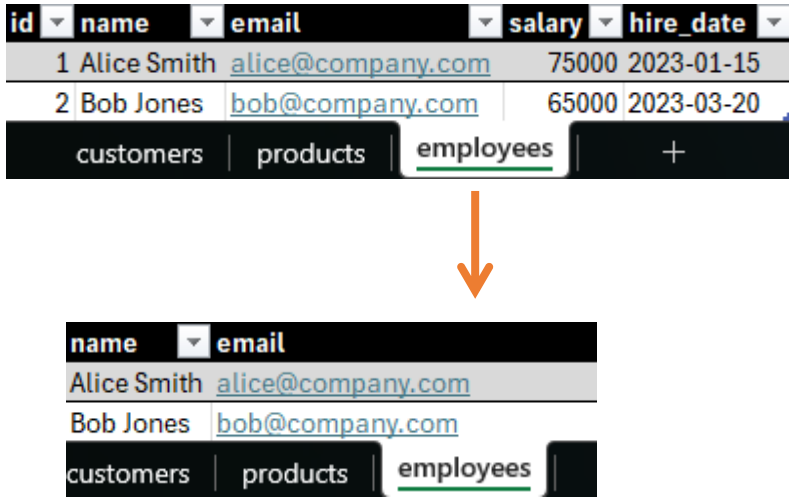
```
(1, 'Alice Smith', 'alice@company.com', 75000, '2023-01-15')
(2, 'Bob Jones', 'bob@company.com', 65000, '2023-03-20')
```

Querying Data with SELECT

The **SELECT** statement retrieves data from a table. One of the most important SQL commands.

SQL Syntax – Get specific columns

```
SELECT name, email FROM employees
```



Python Example

```
cursor.execute("SELECT name, salary FROM employees")
results = cursor.fetchall()

for row in results:
   print(row)

**Output:**
('Alice Smith', 75000)
('Bob Jones', 65000)
```

Fetch methods: All, One, and Many

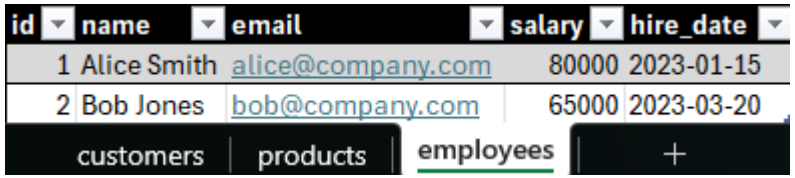
```
cursor.execute("SELECT name, salary FROM employees")
```

`results = cursor`	Return Type	Example
`.fetchall()`	List of tuples	`[('Alice', 75000), ('Bob', 65000)]`
`.fetchone()`	Single tuple or None	`('Alice', 75000)` or `None`
`.fetchmany(n)`	List of tuples	`[('Alice', 75000), ('Bob', 65000)]`

Updating Data with UPDATE

SQL Syntax

```
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition
```



id	name	email	salary	hire_date
1	Alice Smith	alice@company.com	80000	2023-01-15
2	Bob Jones	bob@company.com	65000	2023-03-20

customers | products | employees | +

Use a WHERE clause to specify which rows to update. Without it, you will modify all rows in the table.

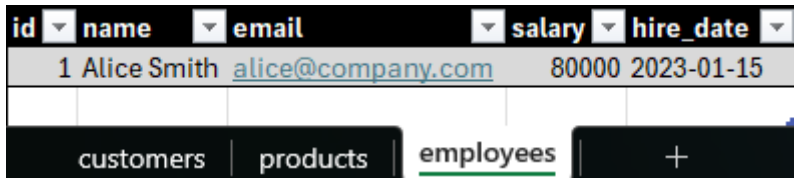
Python Example

```
import sqlite3

conn = sqlite3.connect('company.db')
cursor = conn.cursor()

# Update Alice's salary to 80000
cursor.execute("""
    UPDATE employees
    SET salary = 80000
    WHERE name = 'Alice Smith'
""")

conn.commit()
conn.close()
```

Deleting Data with DELETE FROM

SQL Syntax

```
DELETE FROM table_name
WHERE condition
```



Always use a WHERE clause to specify which rows to delete. Without it, you will delete all data in the table and it cannot be recovered.

DO NOT DO THIS!
cursor.execute("DELETE FROM employees")
This deletes EVERY row in the table permanently!

Python Example

```
import sqlite3

conn = sqlite3.connect('company.db')
cursor = conn.cursor()

Delete the employee with id = 2
cursor.execute("""
   DELETE FROM employees
   WHERE id = 2
""")

conn.commit()
conn.close()
```

From Raw SQL to Classes to SQLModel

Review: Raw SQL in Python

```
import sqlite3
conn = sqlite3.connect("company.db")
cursor = conn.cursor()
# Insert one record
cursor.execute("""
   INSERT INTO employees (id, name, email, salary, hire_date)
   VALUES (1, 'Alice Smith', 'alice@company.com', 75000, '2023-01-15')
""")
# Insert another record
cursor.execute("""
   INSERT INTO employees (id, name, email, salary, hire_date)
   VALUES (2, “Bob Jones”, “bob@company.com”, 65000, “2023-03-20”)
""")
conn.commit()  # Save all changes
conn.close()
```

- What happens if you run this script twice?
- Can you spot an inconsistency between the two INSERT statements?
- What if the name came from user input instead of being hardcoded?
- How many things do you need to keep in sync when writing an INSERT like this?

A typical Class

```
class Employee:
    def __init__(self, name: str, email: str, salary: float):
        self.name = name
        self.email = email
        self.salary = salary

    def __repr__():
                print('Employee(name='Alice Smith', email='alice@company.com', salary=75000)')

    def __eq__(self, other):
                return self.name == other.name

    def monthly_salary(self) -> float:
        return self.salary / 12

if __name__ == "main":
                alice = Employee("Alice Smith", "alice@company.com", 75000)
                print(alice.name)           # Alice Smith
                print(alice.monthly_salary())  # 6250.0
```

Dataclasses: Less Boilerplate

```python
from dataclasses import dataclass

@dataclass
class Employee:
    name: str
    email: str
    salary: float

    def monthly_salary(self) -> float:
        return self.salary / 12

alice = Employee("Alice Smith", "alice@company.com", 75000)
print(alice)  # Employee(name='Alice Smith', email='alice@company.com', salary=75000)
```

What did we NOT have to write compared to the previous version?

Dataclasses: Less Boilerplate

In Python, a **dataclass** is a decorator (`@dataclass`) that automatically generates special methods like `__init__()`, `__repr__()`, `__eq__()`, and more for classes that are primarily used to store data. It reduces boilerplate code and makes your classes cleaner. To generate methods such as __lt__ for a total ordering, @dataclass(order=True) can be used or @total_ordering with any class.

```python
from dataclasses import dataclass

@dataclass
class Person:
    name: str
    age: int
    city: str = "Unknown"  # Default value

# Creating instances
p1 = Person("Alice", 30, "Zurich")
p2 = Person("Bob", 25)

# Auto-generated __repr__ and __eq__
print(p1)       # Person(name='Alice', age=30, city='Zurich')
print(p2)       # Person(name='Bob', age=25, city='Unknown')
print(p1 == p2)  # False
```

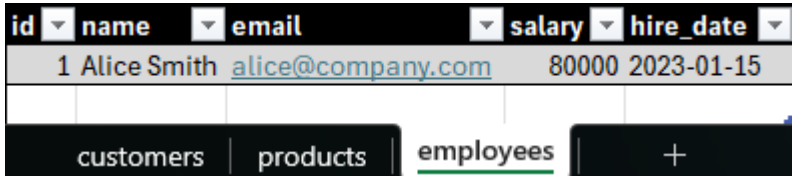
What is an ORM?

ORM stands for Object-Relational Mapping. It connects two worlds:

Python world: classes, objects, attributes
Database world: tables, rows, columns

The mapping: class = table, instance = row, attribute = column.

```python
from dataclasses import dataclass

@dataclass
class Employee:
    name: str
    email: str
    salary: float

    def monthly_salary(self) -> float:
        return self.salary / 12
```



Looking at the Employee class and the employees table, notice the parallels

Raw SQL vs. ORM

With raw SQL

```
cursor.execute("""
   INSERT INTO employees (name, email, salary)
   VALUES (?, ?, ?)""", ("Alice", "alice@company.com", 75000))
conn.commit()
```

With an ORM

```
alice = Employee(name="Alice", email="alice@company.com", salary=75000)
session.add(alice)
session.commit()
```

- Which version would break if we rename *email* to *contact_email*?
- In which version does your editor help you with autocomplete?
- Which version catches a typo before you run the code?

Sidenote: The (?, ?, ?) are placeholders for the variables in the parentheses

Why use an ORM?

- Type safety: your editor catches mistakes before runtime
- Single source of truth: the class defines the structure once
- No raw SQL strings: less room for typos, injection, and quoting issues
- You work with Python objects, not tuples and dicts

Remember the double quotes vs single quotes issue from slide 1? Could that happen with an ORM?

What is SQLModel?

– SQLModel is a library by the creator of FastAPI

– It combines **SQLAlchemy** (database ORM) + **Pydantic** (data validation)

– You define a Python class and SQLModel:

- creates the database table for you
- validates the data types (str, int, float, ...)
- lets you query and save using Python objects

– Think of it as a **dataclass that knows how to talk to a database**

Installing SQLModel

– Install with pip (also installs SQLAlchemy + Pydantic):

```
pip install sqlmodel
```

– Verify the installation:

```
import sqlmodel
print(sqlmodel.__version__)
```

– Works with SQLite out of the box (no extra driver needed).

– Documentation: SQLModel Documentation

Defining a Model

```
from typing import Optional
from sqlmodel import Field, SQLModel

class Employee(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    name: str
    email: str
    salary: float
```

Looks just like a dataclass! But now it maps to a database table.

- `table=True` tells SQLModel this class = a database table
- `id` is auto-generated by the database (primary key)

Creating the Engine & Table

```
from sqlmodel import create_engine, SQLModel

engine = create_engine("sqlite:///company.db")

SQLModel.metadata.create_all(engine)  # creates the employee table
```

- The **engine** is the connection to the database file.
- create_all reads your models and runs CREATE TABLE for you.

Sessions: Talking to the Database

A **Session** is a conversation with the database:

- You add objects, then commit to save them
- Use with block to auto-close the session

```
from sqlmodel import Session

with Session(engine) as session:
    # add, query, update, delete objects here
    session.commit()
```

Adding Data (Create)

```
emp1 = Employee(name="Alice", email="alice@fhnw.ch", salary=95000.0)
emp2 = Employee(name="Bob", email="bob@fhnw.ch", salary=82000.0)

with Session(engine) as session:
    session.add(emp1)
    session.add(emp2)
    session.commit()  # saves both to the database
```

– No SQL needed! Just create Python objects, add to session, commit.

Querying Data (Read)

```
from sqlmodel import select

with Session(engine) as session:
    # get ALL employees
    employees = session.exec(select(Employee)).all()

    # filter: salary > 90000
    query = select(Employee).where(Employee.salary > 90000)
    results = session.exec(query).all()

    # get ONE employee by name
    alice = session.exec(select(Employee).where(Employee.name == "Alice")).first()
```

Updating Data (Update)

– Just change the attribute and commit like editing a variable!

```
with Session(engine) as session:
    # 1. Find the employee
    q = select(Employee).where(Employee.name == "Alice")
    alice = session.exec(q).first()

    # 2. Change the attribute
    alice.salary = 100000.0

    # 3. Add + commit to save
    session.add(alice)
    session.commit()  # runs UPDATE employee SET salary=100000 ...
```

Deleting Data (Delete)

– Find the object, then call session.delete():

```
with Session(engine) as session:
    q = select(Employee).where(Employee.name == "Bob")
    bob = session.exec(q).first()

    session.delete(bob)
    session.commit()  # runs DELETE FROM employee WHERE ...
```

– Same pattern every time: find modify/delete commit.

Relationships (ForeignKey)

```
from typing import Optional

class Team(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    name: str

class Employee(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    name: str
    salary: float
    team_id: Optional[int] = Field(default=None, foreign_key="team.id")

team = Team(name="Engineering")
emp = Employee(name="Alice", salary=85000, team_id=team.id)
```

Project Structure for Models

A clean layout keeps each model in its own file:

```
my_project/
 ├── main.py
 ├── database.py        # engine + create_all()
 └── models/
    ├── __init__.py     # re-exports all models
    ├── employee.py     # class Employee(SQLModel)
    └── team.py         # class Team(SQLModel)
```

models/__init__.py

```
from .employee import Employee
from .team import Team
```

→ Now `from models import Employee, Team` works from anywhere in the project

Exercises

Try these on your own:

1. Create a Department model with id, name, and budget fields
2. Insert 3 departments and 5 employees into the database
3. Query all employees with a salary above 60 000
4. Update an employee's salary and verify the change
5. Delete one employee and confirm they are removed
6. Add a foreign_key linking Employee to Department

Summary

- SQLModel combines **SQLAlchemy** + **Pydantic** in one simple class
- Models are Python classes that map to database tables
- Use Session for all CRUD operations (Create, Read, Update, Delete)
- The select() function replaces raw SQL queries with type-safe Python
- Use foreign_key in Field() to link tables together
- Always use session.commit() to save changes and session.refresh() to reload
- No raw SQL needed: write Python, get SQL for free!